

Lab 1: Customer Support Resolution Agent

Alignment: Domain 1 - Agentic Architecture & Orchestration Estimated time: 75 minutes Mode: Local simulation; no API key required

Objective

Build a bounded agent loop that enforces a refund prerequisite, normalizes tool output, checkpoints state, and escalates high-impact ambiguity.

Scenario

A subscription support team wants Claude to classify a request, inspect account status, propose a remedy, and either close or escalate the case. Refund execution must never occur without an eligibility result and human approval.

Prerequisites

- Python 3.10 or newer.
- A terminal opened in this lab folder.
- No production account, network access, or secret is required.

Architecture before execution

Read the source and identify the control boundary, the evidence artifact, and the terminal condition before running it. All side effects are confined to this lab's evidence/ folder.

Procedure

1. Inspect support_agent.py. Trace the state transitions from CLASSIFY to either RESOLVE or ESCALATE. `bashpython3 -B support_agent.py --demo`###` **2. Run the eligible-account path. Confirm that the agent proposes a refund but records a human-approval requirement instead of executing it.** `bashpython3 -B support_agent.py --case fixtures/eligible.json`###` **3. Run the ambiguous path. Confirm that missing purchase evidence produces a typed escalation with a confidence basis.** `bashpython3 -B support_agent.py --case fixtures/ambiguous.json`###` **4. Inspect evidence/session.json and identify the checkpoint, tool result envelope, stop reason, and idempotency key.** `bashpython3 -m json.tool evidence/session.json`###` **5. Run the verifier and save its output.** `bashmkdir -p evidence && python3 -B verify.py | tee evidence/verification.txt``

Acceptance checks

- [] A maximum-iteration guard terminates the loop.
- [] Refund eligibility is checked before any remedy is proposed.
- [] High-impact uncertainty routes to human review.
- [] The checkpoint contains a stable case ID and idempotency key.

Reflection

Which state transition prevents an irreversible side effect, and why is a prompt-only instruction weaker than this code-level guard?

Evidence and completion

Save terminal output in evidence/verification.txt. The lab is complete when the verifier prints PASS, every acceptance check is satisfied, and your reflection identifies one design trade-off.

Troubleshooting

- Run commands from this lab folder so relative paths resolve correctly.
- Use `python3 -B` to prevent learner machines from creating `__pycache__` artifacts.
- If a check fails, read the named failed assertion, inspect the referenced file, and rerun only after correcting the underlying design.

Clean-up

Remove only files you created outside this folder. Never store API keys, access tokens, customer data, or production logs in lab evidence.